

ПРЕЗЕНТАЦИЯ MVP

BPMN Architect AI

Создание диаграмм BPMN-процессов из текстового описания

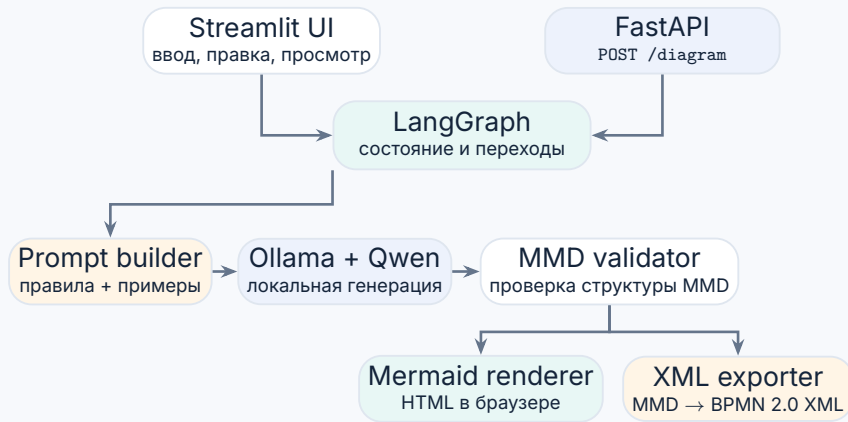
Команда: Углобульб

Лобанов Павел

| Салимли Айзек

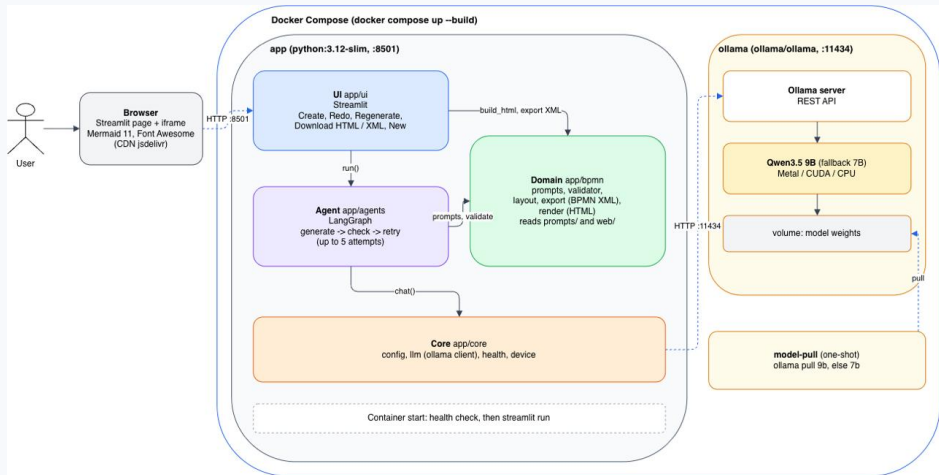
| Черепнов Максим

Логика работы системы



UI и API работают через единый граф агента LangGraph.

Техническая архитектура и развёртывание



Требования для запуска системы

01

Оборудование

- ✓ RAM: 16 ГБ+
- ✓ CPU: 4 ядра+
- ✓ Диск: 15 ГБ+
- ✓ GPU: опционально
CUDA / MPS

02

Программное обеспечение

- ✓ Windows 10/11, Linux, macOS
- ✓ Docker 24+
Compose v2
- ✓ Python 3.12/3.13
и Ollama
- ✓ Браузер:
Chrome, Firefox, Safari, Edge

03

Сеть и порты

- ✓ Интернет при первом запуске (образы + модель)
- ✓ CDN jsdelivr
Mermaid + Font Awesome
- ✓ Порты:
8501 — UI
11434 — Ollama

Стек технологий

01

Язык и окружение

- ✓ Python 3.12
- ✓ Docker + Docker Compose
- ✓ Скрипты запуска:
up.sh / up.ps1

Интерфейс

- ✓ Streamlit 1.65+
- ✓ FastAPI + uvicorn

Визуализация

- ✓ Mermaid 11 (swimlane-beta)
- ✓ BPMN-иконки + Font
Awesome 6.5
- ✓ JavaScript layout
- ✓ CDN jsdelivr

02

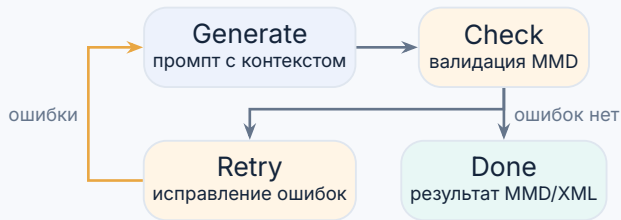
AI

- ✓ Ollama + Python client
- ✓ Qwen3.5 9B
(fallback 7B)
- ✓ LangGraph
- ✓ Metal / CUDA / CPU

Проверка и экспорт

- ✓ Python validator
- ✓ re, dataclasses,
ElementTree
- ✓ BPMN 2.0 XML + BPMNDI

Ключевой компонент — цикл генерация → валидация



ПОЧЕМУ ЭТО НАДЁЖНЕЕ

- ▶ AI генерирует процесс, отдельный модуль проверяет результат.
- ▶ Ошибки автоматически возвращаются в цикл генерации.
- ▶ Некорректные BPMN-схемы не передаются пользователю.

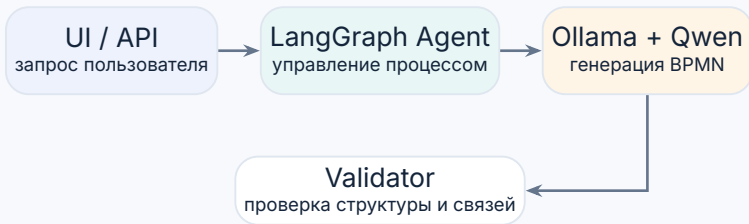
Проверка корректности процесса

ПРОВЕРКИ

- ▶ Наличие всех элементов и связей процесса.
- ▶ Корректность переходов между этапами.
- ▶ Правильность развилок и условий выполнения.
- ▶ Заголовок, дорожки, пулы и объявления элементов.
- ▶ Входящие/исходящие связи, условия XOR и корректное использование gateway.



Архитектура AI-сервиса



УПРАВЛЕНИЕ ПРОЦЕССОМ ГЕНЕРАЦИИ

LangGraph связывает генерацию, проверку и повторные попытки в одном сценарии выполнения.

РАЗВЁРТЫВАНИЕ

Docker Compose запускает приложение и локальную LLM-инфраструктуру.

Преимущества выбранного подхода

01

Управляемая генерация

Модель работает по заданным правилам: доступные элементы, структура и связи контролируются валидатором.

02

Борьба с галлюцинациями

Ошибки валидатора возвращаются в генерацию, после чего схема создаётся повторно.

03

Локальное развёртывание

Решение способно работать внутри инфраструктуры организации без передачи данных внешним AI-сервисам.

Модульная архитектура удобна для дальнейших расширения и поддержки решения

UI: окно запроса построения диаграммы

BPMN Architect

Опишите бизнес-процесс: участники, шаги, условия, результат

Press Ctrl+Enter to apply

Создать

UI: окно ожидания построения диаграммы

BPMN Architect

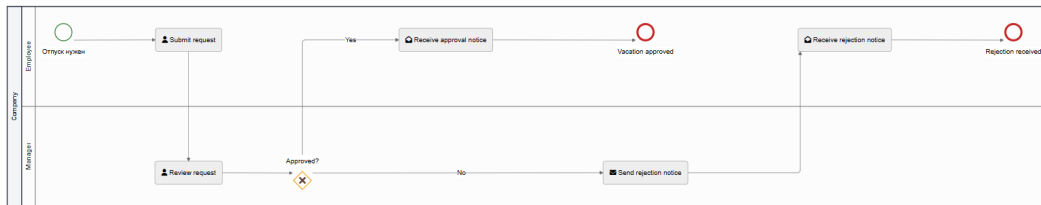
Сотрудник отправляет заявку на отпуск. Менеджер проверяет заявку. Если менеджер одобряет заявку, система уведомляет сотрудника об одобрении и обновляет данные в HR-системе. Если менеджер отклоняет заявку, система уведомляет сотрудника об отклонении. Процесс завершается.

Создать

☐ Строю диаграмму...

Пример: сгенерированная диаграмма BPMN

Сотрудник отправляет заявку на отпуск. Менеджер проверяет заявку. Если заявка одобрена, система уведомляет сотрудника и обновляет данные в HR-системе. Если заявка отклонена, система уведомляет сотрудника об отказе. Процесс завершается.



Пример: XML код диаграммы BPMN

01 Структура BPMN

```
<bpmn:process id="Process_1"
  name="Company">

  <bpmn:laneSet id="LaneSet_1">

    <bpmn:lane id="L1"
      name="HR System">

      <bpmn:flowNodeRef>
        s1
      </bpmn:flowNodeRef>

      <bpmn:flowNodeRef>
        t1
      </bpmn:flowNodeRef>

      <bpmn:flowNodeRef>
        g1
      </bpmn:flowNodeRef>

    </bpmn:lane>

  </bpmn:laneSet>

</bpmn:process>
```

02 Элементы процесса

```
<bpmn:startEvent id="s1"
  name="Vacation request received">

  <bpmn:serviceTask id="t1"
    name="Check request">

    <bpmn:exclusiveGateway id="g1"
      name="Approved?">

      <bpmn:sendTask id="t2"
        name="Notify approval">

        <bpmn:subProcess id="sp1"
          name="Update HR data">

          <bpmn:endEvent id="e1"
            name="Process finished">
```

03 Связи и DI

```
<bpmn:sequenceFlow id="Flow_1"
  sourceRef="s1"
  targetRef="t1"/>

<bpmn:sequenceFlow id="Flow_2"
  sourceRef="t1"
  targetRef="g1"/>

<bpmn:sequenceFlow id="Flow_3"
  sourceRef="g1"
  targetRef="t2"/>

<bpmndi:BPMNShape id="g1_di"
  bpmnElement="g1">

  <dc:Bounds
    x="490"
    y="30"
    width="50"
    height="50"/>

</bpmndi:BPMNShape>
```

Текущая версия прототипа и развитие проекта

Реализовано

- ✓ Генерация BPMN-схем из текста
- ✓ Проверка структуры и потоков
- ✓ Автоматическое исправление ошибок
- ✓ Конвертация Mermaid MMD → BPMN 2.0 XML
- ✓ Веб-приложение и API

Следующие шаги

- 01 Тесты и метрики качества
- 02 Расширение проверки BPMN-логики
- 03 Интеграция с корпоративными системами